

Chapter 7

Seeing the World as the Shell Sees It

Expansion & Quoting — Study Notes

Every time you press ENTER, **bash** rewrites your command line *before* running it. This rewriting is called **expansion**, and controlling it is called **quoting**. The chapter introduces only one new command — **echo** (print text arguments to standard output) — and uses it to expose all the invisible work the shell does. The book argues these are among the *most important* topics in the whole shell: without them, the shell stays a source of mystery.

1 Expansion: The Core Idea

echo simply prints its arguments. Yet **echo *** does not print an asterisk — it prints the filenames in the current directory:

```
$ echo *
Desktop Documents ls-output.txt Music Pictures Videos
```

What Happened

The shell **expanded** the ***** into the matching filenames *before* **echo** ran. **echo** never saw the ***** — only its expanded result. This is the pattern behind everything in the chapter: you type something, the shell transforms it, and the command receives the transformed result.

2 Pathname Expansion (Globbing)

The mechanism behind wildcards. The shell consults the *filesystem* and replaces the pattern with matching pathnames.

```
$ echo D*                # names beginning with D
Desktop Documents
$ echo *s                # names ending in s
Documents Pictures Templates Videos
$ echo [[:upper:]]*      # names beginning with uppercase
Desktop Documents Music Pictures Public Templates Videos
$ echo /usr/*/share      # works with full paths too
/usr/kerberos/share /usr/local/share
```

An Unmatched Glob Is Left Literal

If a pattern matches *nothing*, the shell (by default) passes the pattern through unchanged rather than expanding to empty. So **echo b*** with no matching files prints the literal text **b***. This is why **rm b*** with no matches fails with “cannot find **b***” — the literal pattern reached the command.

Hidden Files

Names beginning with a period are hidden, and pathname expansion respects this: `echo *` does *not* reveal them. Naively using `echo .*` can wrongly include `.` (current dir) and `..` (parent dir). A safer pattern is `.[!.]` (one leading period followed by non-period characters), or simply use `ls -A` (“almost all”) for a correct hidden-file listing.

3 Tilde Expansion

At the start of a word, `~` expands to a home directory — the current user’s, or a named user’s.

```
$ echo ~  
/home/me  
$ echo ~bob  
/home/bob
```

4 Arithmetic Expansion

Form: `$((expression))`. Turns the shell into an integer calculator.

```
$ echo $((2 + 2))  
4
```

Operators (Integer Only)

<code>+ - *</code>	addition, subtraction, multiplication
<code>/</code>	division — integer only , so <code>5/2</code> gives <code>2</code>
<code>%</code>	modulo (remainder) — <code>5%2</code> gives <code>1</code>
<code>**</code>	exponentiation — <code>5**2</code> gives <code>25</code>

Spaces are insignificant, expressions may be grouped with parentheses, and expansions may be nested:

```
$ echo $(((5**2) * 3))  
75  
$ echo Five divided by two equals $((5/2))  
Five divided by two equals 2
```

5 Brace Expansion

Creates multiple strings from one pattern. A brace expression holds either a **comma list** or a **range** (`..`); text before it is the *preamble*, text after is the *postscript*.

```
$ echo Front-A,B,C-Back  
Front-A-Back Front-B-Back Front-C-Back  
$ echo Number_1..5
```

```
Number_1 Number_2 Number_3 Number_4 Number_5
```

Ranges accept zero-padding and reverse order, and braces may be nested:

```
$ echo 01..15
01 02 03 04 05 06 07 08 09 10 11 12 13 14 15
$ echo Z..A
Z Y X W V U T S R Q P O N M L K J I H G F E D C B A
$ echo aA1,2,B3,4b
aA1b aA2b aB3b aB4b
```

Rules & Best Use

The pattern may not contain **unquoted whitespace**. The most common use is generating lists of files or directories to create — the whole set in a single command:

```
$ mkdir 2007..2009-01..12 # 36 Year-Month directories at once
```

Zero-padding (**01** not **1**) makes the names sort correctly, because names sort as *text*, character by character — unpadded, **10** would sort right after **1**.

6 Parameter Expansion

Reveals the contents of a named variable with **\$**. Touched on only briefly here; far more useful in scripts.

```
$ echo $USER
me
$ printenv | less # list available variables
```

A Misspelled Variable Fails Silently

Unlike a mistyped glob (which prints the literal pattern), a mistyped *variable* name still expands — to an **empty string**, with no error:

```
$ echo $SUER
# blank line -- nothing printed
```

This invisible failure is dangerous in scripts: **rm -rf \$TYP0/data** becomes **rm -rf /data**.

7 Command Substitution

Uses the *output* of a command as an expansion. Modern form **\$(command)**; older form uses back-quotes **`command`**.

```
$ echo $(ls)
Desktop Documents ls-output.txt Music Pictures Videos
$ ls -l $(which cp) # output of `which cp` becomes an argument
```

```
-rwxr-xr-x 1 root root 71516 /bin/cp
```

Whole pipelines may be substituted; their result becomes the argument list of the outer command:

```
$ file $(ls -d /usr/bin/* | grep zip)
```

Substitution vs. Pipe

A **pipe** (`|`) feeds output into another command's *standard input* — a stream to be read. **Command substitution** pastes output onto the command line as *arguments* — as if you had typed them. `echo` ignores stdin, so `ls | echo` prints blank, but `echo $(ls)` prints the filenames.

8 Quoting: Controlling Expansion

Two failures motivate quoting: word-splitting collapses whitespace, and an undefined variable becomes an empty string.

```
$ echo The total is $100.00
The total is 00.00      # $1 was undefined -> empty; "00.00" remained
```

Double Quotes — Suppress Most, Keep the \$ Expansions

Inside `"~"`, special characters lose meaning *except* `$`, `\`, and ```. So **word-splitting**, **pathname**, **tilde**, and **brace** expansion are suppressed, but **parameter**, **arithmetic**, and **command substitution** still run.

```
$ ls -l "two words.txt"  # quotes bind the space into one argument
$ echo "$USER $((2+2)) $(echo hi)"
me 4 hi                  # the three $-expansions still fire
```

Word-Splitting, Precisely

By default the shell treats **spaces**, **tabs**, and **newlines** as delimiters — they separate text into arguments and are otherwise discarded. Double quotes stop this, so the whitespace becomes part of a single argument. This produces a subtle effect on command substitution:

```
$ echo $(df -h)          # unquoted: 49 separate arguments, one flat line
$ echo "$(df -h)"        # quoted: ONE argument, newlines preserved
```

The Key Count

Unquoted substitution → word-splitting runs → *many* arguments → `echo` rejoins them with single spaces (looks flattened). Quoted substitution → no splitting → *one* argument with spaces and newlines intact (original layout preserved). More splitting means more arguments *and* more flattening.

Single Quotes — Suppress Everything

To suppress *all* expansions, use single quotes. The comparison across levels:

```
$ echo text ~/.txt a,b $(echo foo) $((2+2)) $USER
text /home/me/ls-output.txt a b foo 4 me
$ echo "text ~/.txt a,b $(echo foo) $((2+2)) $USER"
text ~/.txt a,b foo 4 me
$ echo 'text ~/.txt a,b $(echo foo) $((2+2)) $USER'
text ~/.txt a,b $(echo foo) $((2+2)) $USER
```

Expansion vs. Quoting Level

Expansion	none	"~"	'~'
Tilde ~	on	off	off
Pathname *	on	off	off
Brace {a,b}	on	off	off
Parameter \$USER	on	on	off
Arithmetic \$((...))	on	on	off
Command sub \$(...)	on	on	off
Word-splitting	on	off	off

Each level suppresses more. The double-quote shortcut: “does it start with \$?” — if so it survives.

Escaping Characters

To quote a *single* character, precede it with a backslash — the **escape character**. Common inside double quotes to block one expansion:

```
$ echo "The balance for user $USER is: $5.00"
The balance for user me is: $5.00
```

Also used to strip the special meaning of characters in filenames, and to write a literal backslash as `\\`:

```
$ mv bad&filename good_filename
```

Within single quotes the backslash is itself an ordinary character. A backslash before a command also suppresses an *alias* (e.g. `\ls` runs raw `ls`, ignoring a color alias).

Backslash Escape Sequences

The backslash also encodes control codes. `echo` interprets them only with the `-e` flag (or inside `$'...'`).

Common Sequences

<code>\a</code>	bell (alert beep)
<code>\b</code>	backspace
<code>\n</code>	newline (linefeed)
<code>\r</code>	carriage return
<code>\t</code>	tab

```
$ echo -e "hello"    # -e enables interpretation
hello
bye
$ sleep 10; echo -e "Time's up"
```

Two Independent Controls

Quoting governs shell expansions and word-splitting. The **-e flag** governs whether **echo** turns `\n` into a real newline. They are separate: `echo "a\nb"` prints `a\nb` in *any* quotes without **-e**. Also: `\` (backslash) is the escape character; `/` (forward slash) is just an ordinary character.

9 The Expansion Pipeline (Order Matters)

What the Shell Does on ENTER

1. Brace expansion `{a,b}`
2. Tilde expansion `~`
3. Parameter, arithmetic, command substitution `$USER, $((...)), $(...)`
4. **Word-splitting** — results split into arguments at whitespace
5. Pathname expansion `*`
6. Quote removal, then the command runs

Word-splitting (step 4) runs *on the output of* the earlier expansions — which is exactly why unquoted command substitution gets mangled, and why quoting (which disables step 4) fixes it.

10 Summing Up

Expansion is the shell generating or substituting text; quoting is the dial that decides how much of that interpretation to allow — from “expand everything” (no quotes), through “expand only the `$` forms” (double quotes), to “expand nothing” (single quotes), with the backslash escaping one character at a time. A proper grasp of both is what turns the shell from a source of confusion into a tool that does exactly what you intend.